Development Documents

Project Information

Name: Nina Zahrai 0011
Target Grade: 4
Repository Link: https://github.com/niinzah/Lab2Javascript

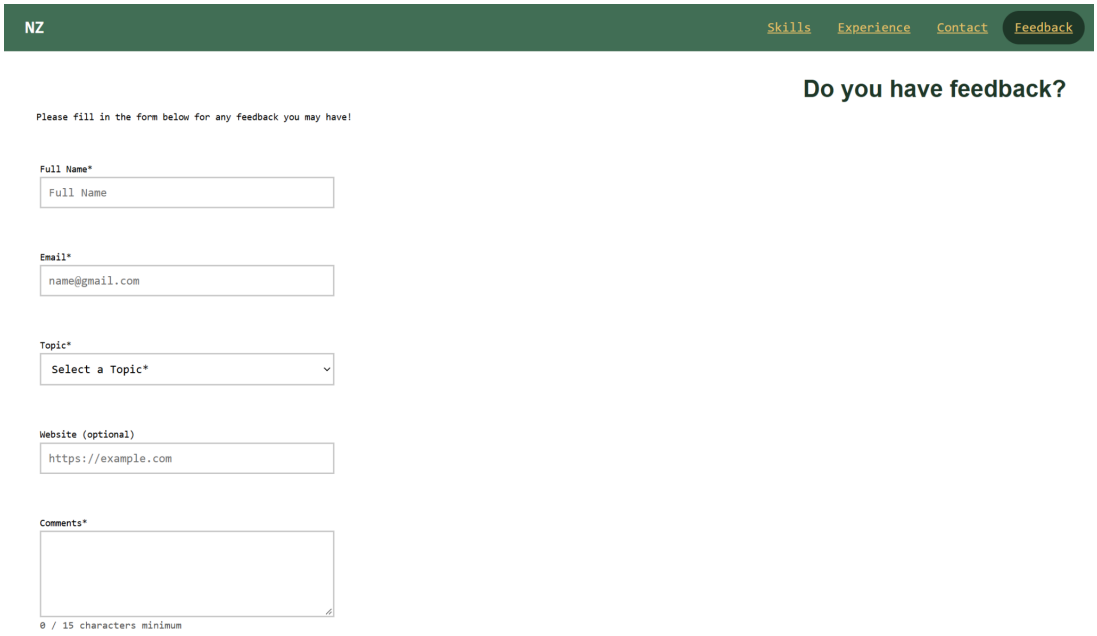
Implemented features:

- feedback form with live counter, error messages if all the necessary information was not filled in, message at the end when form was sent.
- Skill cards that can be filtered and sorted.

Feature Evidence

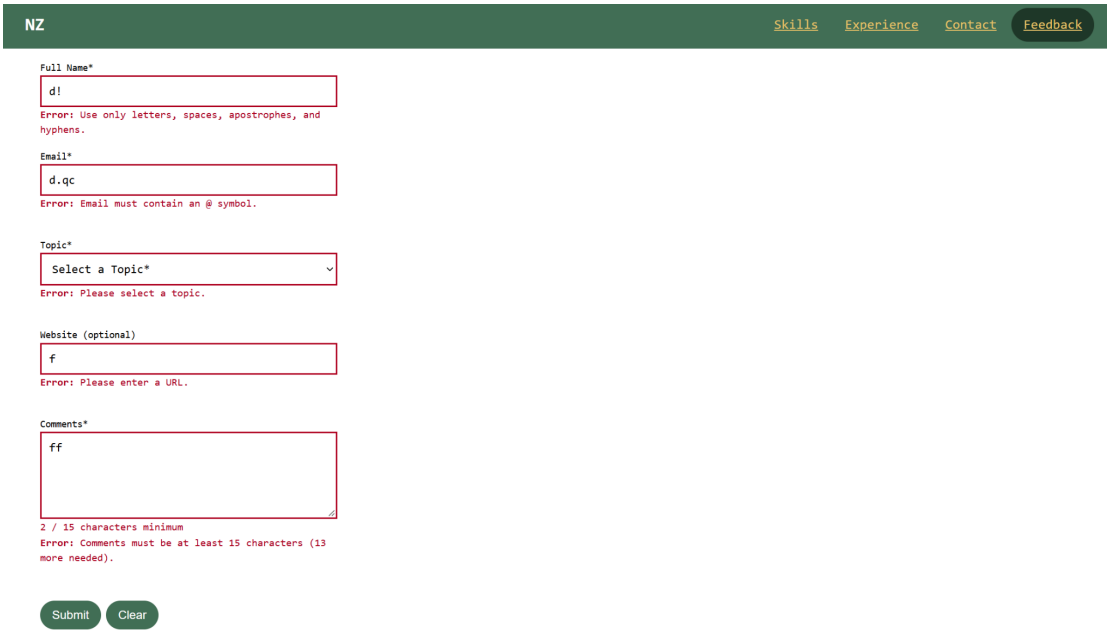
Feedback Form

Normal



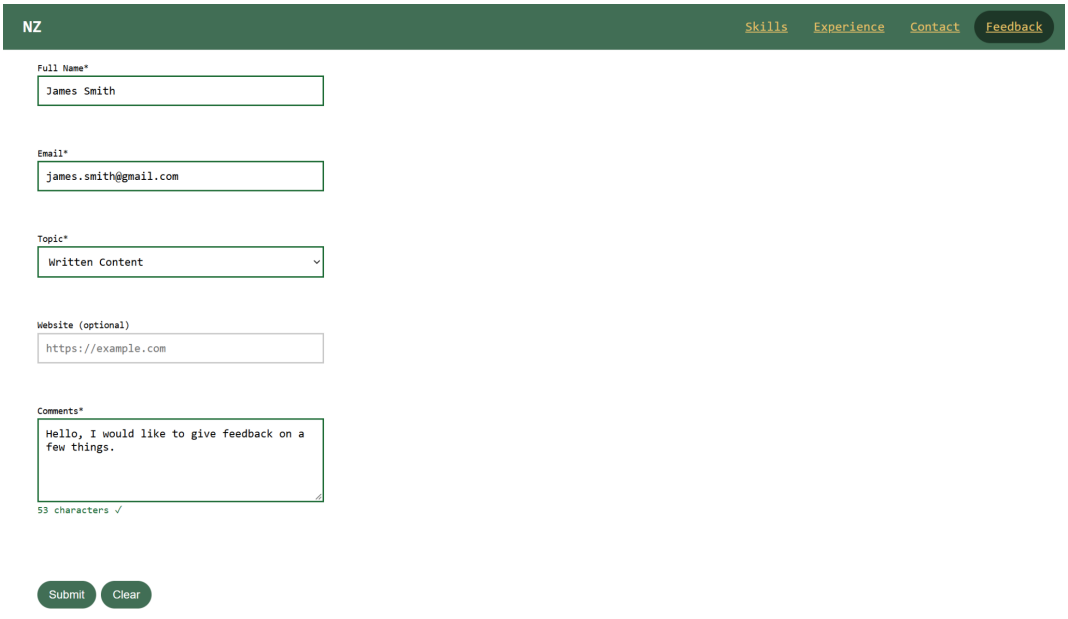
Before filling in the feedback, the placeholders and 0/15 character minimum can be seen.

Errors



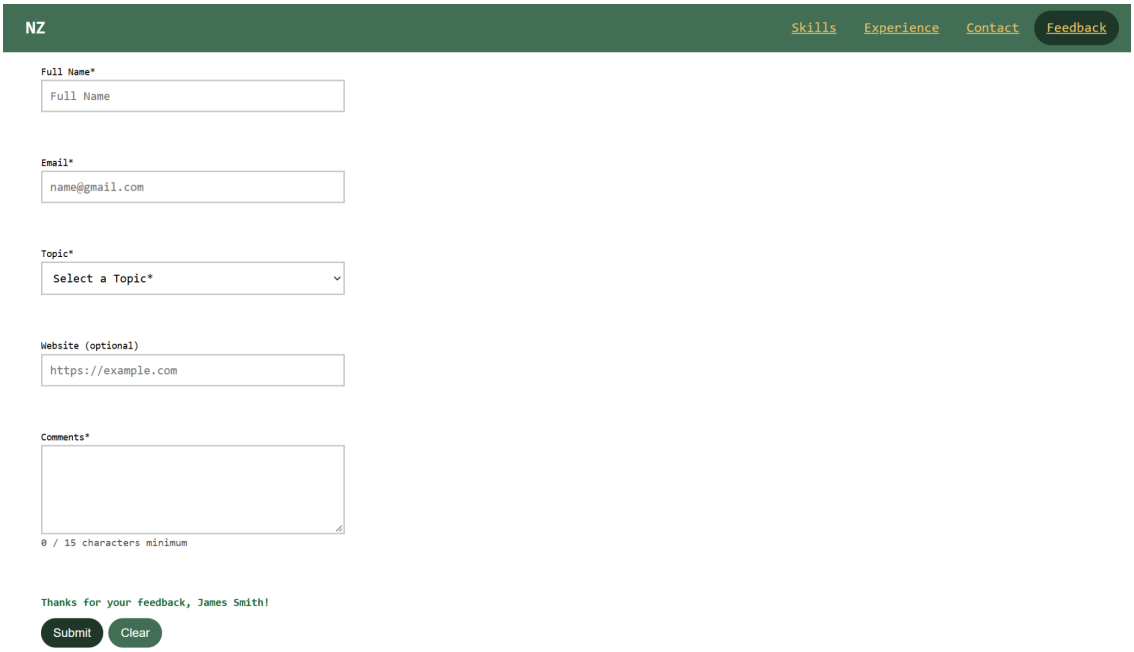
Once filled in, but with the wrong formatting, the error messages can be seen, it is also not possible to submit the form until everything has been fixed.

Valid



When the formatting has been fixed, everything turns green and the error messages disappear.

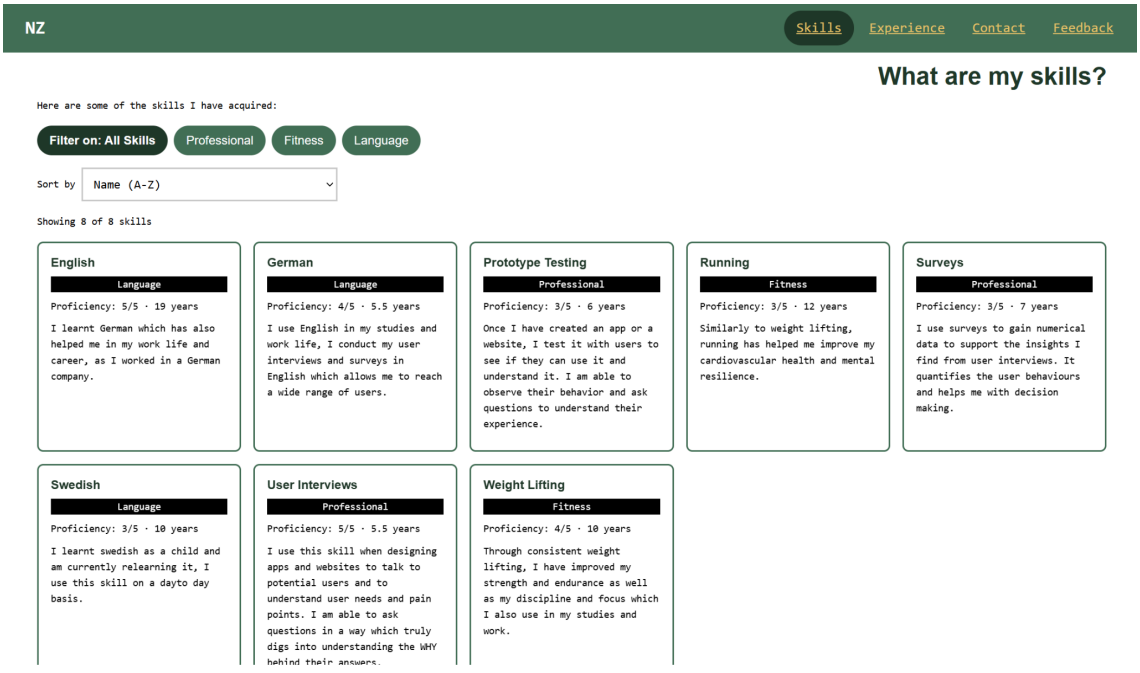
Submitted



Once submitted, a message appears at the end with the name that the user had filled in.

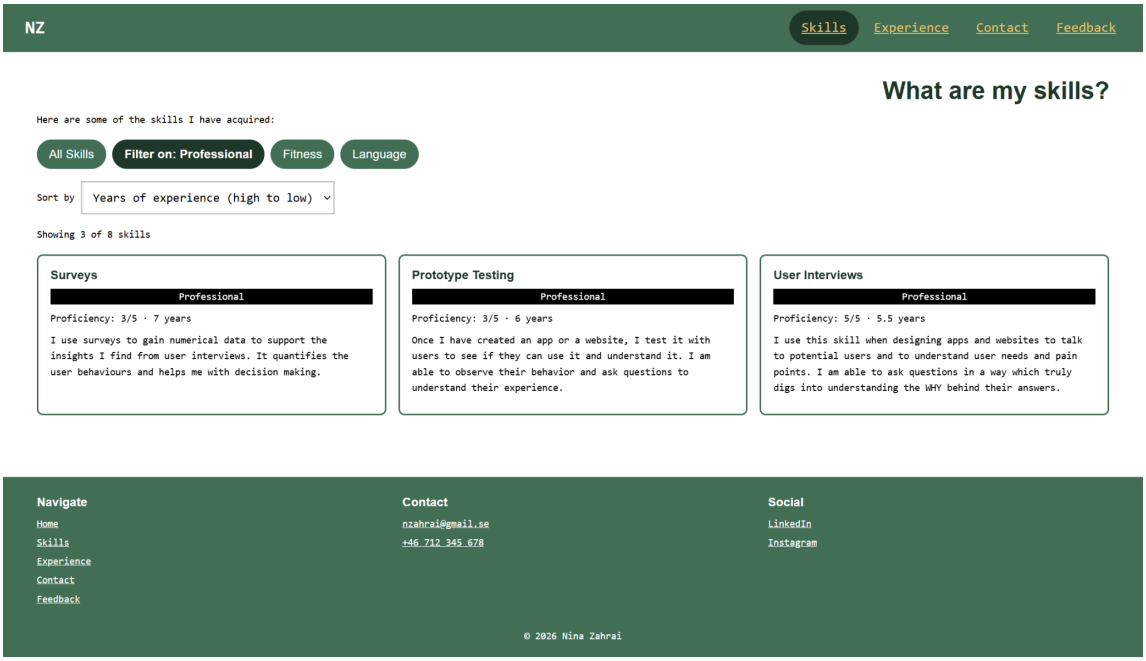
Skills

Normal



For the skills page, without selecting anything, the “all skills” filter is automatically selected and it is sorted alphabetically.

Filter and sort applied



When users select a filter and another sort, both are applied at the same time.

Code Explanation

Feedback Form

The main functions consist of:

- validateFullName()
- validateEmail()
- validateTopic()
- validateWebsite()
- validateComments()
- applyValidation()
- showFieldError()
- clearFieldError()
- setFormStatus()
- resetForm()

When the user clicks on a field then leaves it, that field's validator is called. It reads the value and then returns an error message (or an empty string if it is valid). applyValidation() passes the result to showFieldError(), which writes the message into the error span and sets aria-invalid="true", this then shows as a red border around the field input. On the other hand, a

valid field (i.e. with the right formatting) gets a green border. e.preventDefault() stops the page reloading and all fields are validated. Eventually, either the first invalid field receives focus or setFormStatus() shows “Thanks for your feedback, (name)!” for 4 seconds. The Clear button initiates the reset event, which calls resetForm() to return the page to its normal state.

Skills

The skills array holds 8 objects with the same six fields (id, name, category, proficiency, yearsExperience, description) across 3 categories. It is the only source of the displayed content (this means that none of it was written in html).

renderSkills(list) empties the container, loops through the list and creates one .skill-card for each skill, then adds each card to the page.

Clicking a filter button initiates a click event, and the button’s data-category is then passed to filterSkills(), which saves it in currentCategory. Changing the dropdown activates a change event, which saves the value in currentSort. Both then call updateSkills(), which filters the array, sorts a copy, calls renderSkills(), updates the “Showing X of 8 skills” text.

Problem Solving

Problem 1: no error messages

In an initial stage it was possible to submit the feedback form with empty fields (even in the required ones). The page would reload and the error messages would disappear. To understand where the issue was, I checked the validation function and the “if” condition and added a console.log line to see which parts were functioning. I found that the script crashed before reaching preventDefault(), because the HTML had no topicError element, so topicError.textContent threw an error on null. Adding the missing element fixed it. I also moved e.preventDefault() to the first line of the submit handler, so the page can never reload.

Problem 2: filter and sort would not work together

The filtering and sorting options would not work together. This was because the page had category filters and a sort dropdown, but each control built its own list, so choosing a sort after a filter brought back all eight cards (or the reverse), and the buttons no longer matched what was on screen. I fixed it by storing the user’s choices in two shared variables, currentCategory and currentSort, and moving the logic into one updateSkills() function. That function filters the skills array, sorts a copy with [...filtered].sort(...) so the original data is never reordered, then calls renderSkills() to rebuild the cards. It also updates the “Showing X of 8 skills” text and sets aria-pressed on the buttons. Now each event handler only updates its own variable and calls updateSkills(), so the filter, sort and display can never become out of sync.

Reflections

I feel the most comfortable describing the input fields in feedback form as it is the part that is the most intuitive to me. It is straight-forward to explain that a validator reads a value and returns an error message, or an empty string if the value is valid. Expectedly, these validators vary based on the requirement of the input field.

My Git History shows a step by step process of implementing each part. For instance, I first added the alphabetical sorting in the skill card section, then I added the proficiency and so on. I could have been better with pushing commits even when things were going wrong, as I only did so for parts in when I had resolved the problems, in this way it would have been easier to remember and refer back to issues I had faced.

If I had one more development session, I would add another feature where the experience page would be a bit more interactive, perhaps they would have different viewing options. Either people could view it as a timeline as it is now, or they could see it in categories (similar to the skills section), or a CV. Within the CV people could click on an experience they found more interesting and read more about.